

Model-Grounded Interaction Contracts for Traceable Spatial Agent Interfaces in Unity

ANONYMOUS AUTHOR(S)

People in interactive three-dimensional environments increasingly communicate with embodied agents about objects that they can see, approach, or point to. Implementing such an interface requires developers to keep scene selections, stable entity identities, agent responsibilities, runtime actions, and validation evidence consistent. These relationships are commonly distributed across a scene editor and an agent backend, which makes interaction failures difficult to explain and changes difficult to validate. We present FunctionalMLDS, a Functional Meta-Language-defined Structure and model-grounded interaction-assurance layer for embodied multi-agent interfaces, implemented in a Unity desktop prototype and a conversational-agent backend. The layer makes the path from a user-facing scenario and spatial target to an agent capability, runtime action, and validation assertion explicit and executable. An admitted request must resolve that entire path in one pinned model version; a failed request is assigned to the first violated relation instead of being treated as a generic chat failure. We evaluate the approach against a normalized adapter over the corresponding direct artifacts in three spatial environments, using exhaustive structural probes over a fixed authored corpus, contract tests, and systematic fault injection. All 93 assets had a complete declared model-grounded interaction chain (186/186 target-specific step–capability pairs); these are static model records, not 186 executed conversations. Under a shared deterministic resolver, the two adapters agreed on all 93 responsibility and 459 routing decisions and detected and localized all 9 common injected faults. An exhaustive runtime sweep admitted all 298 local or direct routes with the expected target and provider and cross-field-consistent binding identifiers, then rejected all 161 transitive or unreachable routes before the response stub without session mutation. A multi-scenario Unity smoke loaded 31 scenario actions. The contribution is a technical and interaction-oriented account of how explicit model relations can authorize, diagnose, and reproducibly test spatial-agent behavior; it does not claim improvements in subjective user experience without a human-participant study.

CCS Concepts: • **Human-centered computing** → **Intelligent user interfaces**; *Interactive systems and tools*; • **Computing methodologies** → *Intelligent agents*.

Additional Key Words and Phrases: intelligent user interfaces, embodied agents, spatial grounding, model-driven authoring, runtime traceability, Unity

ACM Reference Format:

Anonymous Author(s). 2026. Model-Grounded Interaction Contracts for Traceable Spatial Agent Interfaces in Unity. 1, 1 (July 2026), 19 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

1 Introduction

Imagine a visitor standing in a virtual classroom, pointing at a skeleton, and asking an embodied agent, “What is this?” The visible object, the visitor’s selection, the responsible agent, and the relevant knowledge are all part of the interaction. The system must resolve the object, select the declared agent and capability, and expose ambiguity or failure instead of guessing.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

This apparently simple interaction crosses two development worlds. Unity contains rooms, zones, assets, colliders, avatars, and interaction components. The agent backend contains personas, knowledge, routing rules, and model calls. Direct wiring distributes their relationships across files and code. An endpoint can succeed while the answer concerns the wrong object or comes from the wrong agent, and scene changes can leave stale links hidden until runtime.

We investigate whether an explicit model-grounding layer can make these relationships executable and observable. FunctionalMLDS expands to *Functional Meta-Language-defined Structure*. It builds on the Meta-Language-defined Structure (MLDS) idea of a machine-readable artifact governed by a meta-language specification [14], but makes functional interaction relations first-class. Our system represents user-facing scenarios, spatial targets, embodied agents, capabilities, runtime bindings, expected assertions, and observed outcomes in a shared interaction model. A generation pipeline materializes this model in a Unity desktop client and a multi-agent backend. Unity supplies a ray-selected target candidate; the contract validates its stable identity, determines the declared provider and route, and permits a model call only after those checks pass. Runtime events carry model identifiers back into a validation layer, allowing a developer to follow an interaction from a user request to its outcome.

We call an interaction *interaction-model-grounded* when the selected entity, responsible provider, invoked capability, concrete runtime action, and expected assertion all resolve from the same pinned model instance. This is a conditional assurance guarantee: if the backend admits a spatial request, that request has one complete, authorized model path; otherwise it fails before session mutation or answer generation with the earliest violated relation identified. The guarantee concerns identity, authority, and trace coherence. It does not establish that the selected object matches an unobserved user intention or that a generated answer is factually correct.

This problem connects areas usually treated separately: embodied agents [10, 20], situated reference [15, 22], conversational XR [4, 9, 12, 27], and model-based interface engineering [7, 32, 33]. We ask whether their explicit connection can provide a control and evidence boundary for a spatial multi-agent interface, not whether any ingredient is new.

The model is infrastructure for an intelligent interface: it determines what a spatial request refers to, which agent may act, which operation is invoked, and what evidence explains the result. The claimed novelty is the composed assurance boundary, in which the same typed relations constrain generation, backend admission, UI presentation, and diagnostics.

We ask three questions:

RQ1: Representation. Can assets in heterogeneous scenes be represented as provider-coherent interaction chains and resolved to a responsible agent under explicit asset, group, and zone rules?

RQ2: Representation tradeoffs. Under semantically matched executable adapters, how do the direct-artifact and model-grounded representations compare in routing parity, detection and localization of common faults, representation edits, and local execution time?

RQ3: Contract enforcement. Which invalid selections, routes, and traces are rejected before state mutation, and can an author preview, validate, discard, or undo a placement change without breaking the model-to-runtime chain?

This paper makes four contributions:

- (1) a model-grounded interaction-assurance architecture for spatial embodied-agent interfaces, implemented across a Unity desktop prototype and conversational-agent backend;
- (2) an executable interaction chain linking scenarios, targets, agents, capabilities, runtime actions, assertions, and evidence, with staged fail-closed enforcement and relation-level diagnostics;

- (3) a reproducible, API-free structural benchmark over three environments, plus deliberately valid and broken Unity/backend interaction traces; and
- (4) an executable comparison with a normalized adapter over the direct pre-model artifacts, with claims bounded to routing parity, synthetic mutations, representation edits, and descriptive local runtime.

This system evaluation makes no claim about preference, trust, workload, or perceived usability.

2 Related Work

2.1 Human-AI interaction and intelligible control

Human-AI interaction research repeatedly shows that a capable model is not enough: an interface must communicate what the system can do, expose status and uncertainty, support correction, and help people recover from errors [1, 35, 36]. Explanation needs depend on the questions people ask about a system [25], while explanations do not automatically improve joint performance [2]. These findings motivate three design choices in our system: agent responsibility is inspectable, routing is reported with a reason, and unresolved spatial references fail visibly instead of being silently guessed. We do not propose new general HAI guidelines or infer subjective benefit from these mechanisms; we test whether the implementation obeys its declared interaction contract.

2.2 Embodied agents and spatial reference

Embodied conversational agents established how language, gaze, gesture, and a visible body combine in an interface [10]. Museum guides further demonstrate the practical and educational importance of agent behavior in a shared place [3, 20]. Recent generative agents add memory, reflection, and social behavior [31], while multi-agent frameworks coordinate language-model roles through conversation [24]. Those systems motivate flexible agent behavior, but they do not by themselves specify which Unity object an utterance concerns or provide a requirements-to-runtime validation trace.

Situated-language research directly studies this reference problem. Approaches combine recognition of a user’s plan and referring expression [15], connect directions to perceived environments [19], maintain symbolic anchors for human-robot discourse [22], or learn perceptually grounded word meanings for incremental reference resolution [18]. FunctionalMLDS does not introduce another general grounding model. It supplies a system boundary around a chosen selection or grounding mechanism: candidates are bound to stable model entities, ambiguity is explicit, responsibility is constrained, and the resolved entity remains available to runtime assertions.

2.3 Conversational authoring and agents in XR

Several recent systems make language models useful inside interactive 3D environments. LLMR prompts and edits interactive mixed-reality worlds in real time and includes planning, scene understanding, and self-debugging [12]. Tell-XR lets end users author event-condition-action automations conversationally in AR and VR [9]. CUIfy packages the integration of LLM-powered conversational agents in XR [4]. AgentHands studies interactive gestures for spatially grounded XR conversations [27].

These systems rule out a claim that combining Unity or XR with LLM agents is itself novel. Our focus is complementary. We make the generated interaction architecture inspectable across spatial entities, agent responsibilities, capabilities, backend actions, and validation evidence. Compared with LLMR and Tell-XR, the target is not free-form world generation or individual automation rules, but lifecycle-wide consistency of an embodied multi-agent interface. Compared with

CUIfy and AgentHands, the contribution is an authoring and verification layer rather than an integration library or gesture-generation method.

2.4 Model-based interface and XR engineering

Model-based interface development has long addressed the mapping from abstract tasks and domain concepts to concrete interfaces [33]. The Cameleon framework organizes transformations across abstraction levels and deployment targets [7]; UsiXML supports multiple development paths [26]; MARIA describes service-oriented interfaces at multiple abstraction levels [32]; and OpenUIDL supports runtime, omni-channel interfaces [30]. FunctionalMLDS is narrower: its domain is spatial, agentic interaction, and its platform mappings connect Unity entities and events to agent capabilities and backend operations.

Model-driven work also targets immersive applications directly. Campos-López et al. define a domain metamodel enriched with AR representations and interactions with external systems, then generate applications with their ALTER tool [8]. Karre and Reddy define a role-based template for VR requirements, including scene, action, state, and validation concepts [17]. These systems establish that a metamodel can organize XR requirements or application generation. Our narrower concern is whether one instance can remain an executable authorization and diagnostic contract across a selected scene object, a responsible embodied agent, and a backend action.

The MLDS lineage is especially close and fixes the terminology used here. Fischer et al. define a Meta-Language-defined Structure (MLDS) as a machine-readable artifact generated under a domain-specific meta-language specification [14]. Burk and Wienkop already apply that idea to natural-language-to-Unity scene generation, including spatial object placement and optional manual refinement [6]. Burk et al. subsequently formalize modular MLDS instructions, versioned profiles, dependencies, and gatekeeper validation for evolving toolchains [5]. Consequently, neither MLDS-to-Unity materialization, placement, nor MLDSI modularization is claimed as new here. The present contribution starts after materialization: a selected scene entity must resolve through one scenario-specific provider, capability, runtime action, and assertion, and that same chain is enforced during backend admission and response presentation.

RADIANT likewise brings model-driven discipline to multi-agent LLM systems by deriving traceable structure and behavior models from requirements and using formal verification and change-impact analysis [34]. It therefore rules out a broad claim that requirements-to-verification modeling for multi-agent LLM systems is new. Our later-lifecycle focus is narrower: a Unity-selected spatial entity must remain bound to one scenario-specific provider, capability, and executable action during request admission and returned interaction evidence.

The design also builds on requirements traceability: a trace is useful when it helps locate and reason about consequences across artifacts [16], and empirical work shows that developers can benefit from trace links during maintenance and evolution [28]. We therefore evaluate concrete change and fault tasks rather than treating the presence of trace links as evidence of benefit.

2.5 Contracts, runtime verification, and agent provenance

Design by Contract makes obligations explicit as preconditions, postconditions, and invariants [29]; runtime verification evaluates specified properties against observations of an executing system [23]. FunctionalMLDS adopts the enforcement intuition but does not claim formal program verification. Its obligations are cross-layer relations: a request has a pinned model, a resolved entity, one authorized provider, an executable binding, and a response that preserves the relevant identifiers. The backend checks these obligations before a model call or state mutation, while later assertion evidence distinguishes contradiction from a missing observation.

Provenance models such as W3C PROV connect entities, activities, and agents so that derivation can be represented and exchanged [21]. Agent diagnosis has a related need for complete execution context: TraceElephant shows that partial output-only traces obscure failure attribution in multi-agent systems [11]. Our trace is deliberately more limited than a complete reasoning trajectory. It records the contract identifiers and observable outcome needed to locate a failure along the scenario–target–provider–action path, without claiming to expose private chain-of-thought or explain answer semantics.

2.6 Positioning

Prior work addresses model-based interface generation, conversational XR authoring, spatial grounding, embodied agents, and LLM-agent orchestration. FunctionalMLDS instead keeps one interaction contract active after materialization, linking the selected entity and responsible agent to a capability, runtime action, and observed assertion. We evaluate routing correctness, change consistency, fault localization, and trace completeness against a normalized direct-artifact adapter; we do not claim the first spatial-agent system, a new grounding algorithm, or a general UI description language. Appendix Table 6 details the closest comparison.

3 System

The core system is the visitor interaction path and its developer-facing diagnostics. Language models propose semantic content and, after deterministic contract checks pass, a conversational reply. A secondary placement workbench reuses the same identifiers for guarded change. We evaluate the contract, not linguistic output quality.

3.1 Interaction-oriented design goals

The visitor uses the Unity surface to select a visible object, see its highlight and status, and ask an agent about it. Three goals connect this interaction to the contract:

Stable reference. A collider is bound to a model entity and remains the request target while the visitor composes a question.

Visible uncertainty. Missing and ambiguous selections are explicit states that block a model call rather than silently choosing an object.

Auditable responsibility. Provider and route follow declared asset, group, zone, capability, and handoff relations and are returned as diagnostic evidence.

The interaction model is the executable infrastructure for these behaviors; it is not itself the user interface.

3.2 Interaction model

FunctionalMLDS represents an interaction at several linked levels. A *scenario step* captures a user-visible action, such as asking an exhibit agent to describe a dinosaur skeleton. A *capability use* identifies the operation, the responsible provider, and the spatial targets. The capability is connected through a *runtime binding* to a concrete backend action. An *assertion* describes the outcome that should be observable, and a *validation case* links a stimulus to that expected outcome.

The *metamodel* defines classes, associations, cardinalities, and invariants; each room is a conforming instance. V2 conservatively extends a selected EAST-ADL 2.1.12 subset [13]. Imported requirements, use-case, function, and verification concepts retain their specified meaning. The FunctionalMLDS namespace adds scenario flow, capabilities,

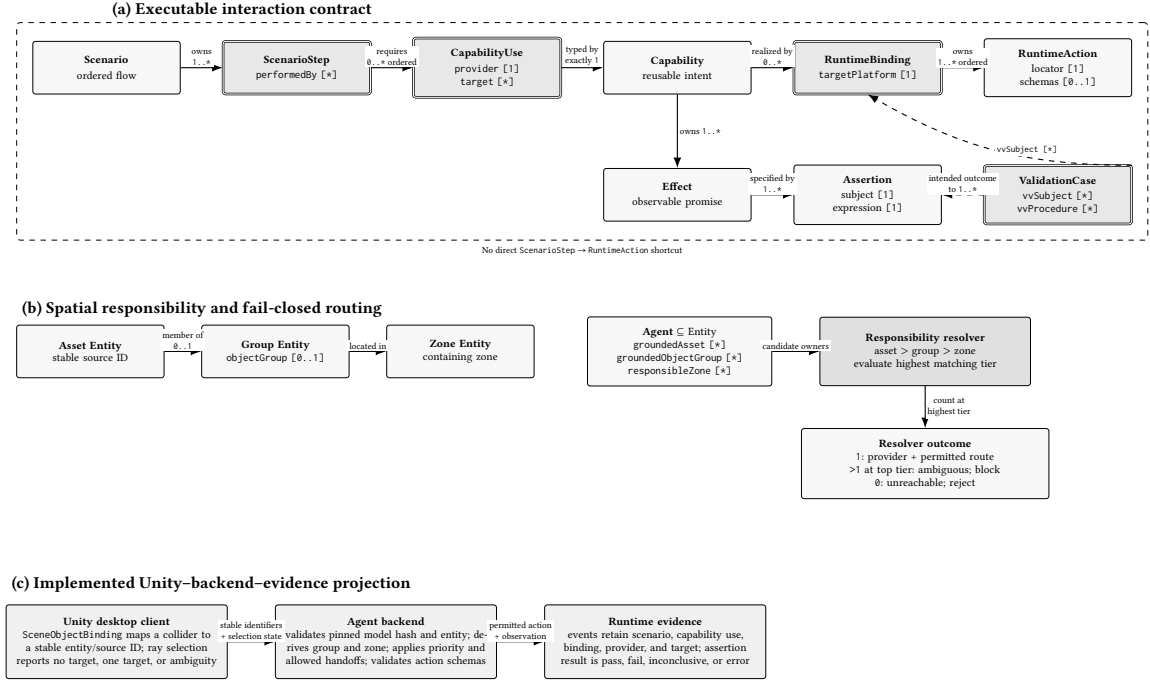


Fig. 1. The compact FunctionalMLDS contract and its implementation mapping. A scenario step reaches a runtime action only through an explicitly provided capability use, capability, and binding. Spatial responsibility is resolved at the highest matching tier—asset before object group before zone—and ambiguity at that tier blocks execution. Stable model identifiers connect the Unity selection, backend routing, and structured assertion evidence.

entities, agents, runtime bindings/actions, assertions, and spatial responsibility and handoff relations. EAST-ADL provides a typed traceability foundation; we do not claim full conformance.

The required execution path is:

ScenarioStep $\rightarrow$ CapabilityUse $\rightarrow$ Capability $\rightarrow$ RuntimeBinding $\rightarrow$ RuntimeAction.

This path intentionally prevents a scenario step from invoking an endpoint without an explicit agent capability. Agents additionally reference their responsible zones and grounded assets. These relationships allow validators to reject a capability use whose provider is not responsible for its spatial target.

Figure 1 shows the model boundary. A CapabilityUse has exactly one capability and one provider in the executable profile. Its targets may include a scene asset, its group, and the provider’s zone. A runtime binding realizes the capability through actions; a validation case connects the binding and resulting assertions to a check. The resolver evaluates asset, then group, then zone ownership. It accepts exactly one owner at the highest matching tier; ambiguity or no owner blocks execution.

In the running example, a visitor selects a dinosaur skeleton and asks “What is this?” The chain resolves the Exhibit Interpreter’s description capability and the chat action; Appendix Table 5 lists the corresponding model objects. Its evidence can verify target, provider, and route metadata, but not the biological accuracy of the generated sentence.

3.3 Staged invariant enforcement

The implementation distinguishes three kinds of rule. *Canonical* rules define model-language well-formedness and backwards-compatible cardinalities. The *executable profile*, enforced while the pipeline assembles a case, tightens those rules where execution needs a unique choice; for example, the compatibility core permits an optional provider, whereas an executable capability use requires exactly one. *Runtime* rules bind a specific request and observation to one already valid model version. Treating all three as “metamodel validation” would hide where a fault is detected.

The enforcement path has five concrete boundaries: JSON schema, canonical model, executable pipeline profile, backend runtime, and Unity client. The backend rechecks the authoritative model relations, so a modified client cannot bypass provider, route, target, or action validation. Appendix Table 7 gives the complete invariant-to-boundary mapping.

There is consequently no endpoint shortcut around the interaction contract in the evaluated path. A scenario step does not store a callable endpoint; the pipeline emits an action only through a valid capability binding; the backend re-resolves the exact action from the pinned model; and Unity presents the answer only when the returned identifiers agree with the pending selection.

3.4 Generation pipeline

From a structured room description, the pipeline normalizes scene entities, derives semantics, proposes agent roles and capabilities, materializes knowledge, and computes placement. A native FunctionalMLDS instance connects these products to scenarios and runtime bindings. Deterministic validators check schemas and references before publication to Unity or the backend.

Language models may propose labels, roles, handoffs, and knowledge. Identifiers, placement, probes, relationship validation, materialization, and metrics are deterministic. Each stage records inputs, configuration, outputs, and hashes so upstream changes invalidate stale results.

3.5 Secondary placement workbench

The Unity editor additionally exposes generated agent placement as a revisioned working copy. An author can inspect stable identifiers, preview a structured change, apply it through deterministic regeneration and validation, and accept, discard, or undo it. Content hashes guard concurrent revision, and failed or discarded changes restore prior bytes. This placement-only transaction is a secondary demonstration that the same model can guard change; it is not the paper’s core contribution or a general natural-language model editor.

3.6 Unity desktop client

The Unity client imports room assets and agent configurations, places embodied agents in their assigned zones, and supports first-person and proximity-based interaction. A scene-object component binds a collider to a stable model entity rather than relying on its GameObject name. In the evaluated desktop path, a camera ray selects that component, the object is highlighted, and the resolved target remains stable while a request is composed. The client sends the pinned model hash, entity and source identifiers, ray-hit metadata, input modality, active agent, and utterance to the backend. No-target and ambiguous states are explicit and do not silently fall through to an LLM call. The bridge does not choose one global action merely by operation kind. It first uses the selected entity to identify the candidate scenario context, then accepts the scenario, capability, binding, and action identifiers confirmed in the server’s `model_binding`.

The corresponding scenario-specific context and runner receive the event. Scene bootstrap likewise requires an exact stable object binding; UUID-suffix variants are not treated as equivalent fallback identifiers.

3.7 Agent backend and handoffs

The backend exposes setup, chat, speech, and project-management endpoints. Each agent has a persona, knowledge scope, capabilities, and permitted handoff targets. A request can be answered by the active agent, delegated to the modeled responsible agent, or rejected when no grounded target or responsible capability exists. The backend validates the model hash and entity before mutating session state. It derives the entity's group and zone from the pinned model rather than trusting client-supplied responsibility fields. Routing gives an explicit asset binding precedence over group and zone responsibility; any handoff is limited to the source agent's modeled targets. Runtime actions carry the interaction-model identifiers needed to distinguish a successful HTTP request from a model-consistent, contract-conforming interaction. When several scenarios share an operation kind, the backend selects by trusted target and routed provider and returns the resulting binding. Zero or multiple exact mappings fail closed; endpoint kind alone is never the dispatch key.

3.8 Runtime validation

Events are written against the originating scenario, capability use, binding, agent, and spatial target. Validation separates four questions:

- (1) Was a syntactically valid runtime operation invoked?
- (2) Was the intended spatial entity resolved?
- (3) Was the responsible agent and capability selected?
- (4) Did the observable response preserve the expected entity and permitted route?

This separation avoids treating network success or a single token overlap as evidence of grounded interaction. It also provides a failure location that can be surfaced in authoring tools or logs.

3.9 Use of machine intelligence

The system uses language models for semantic scene interpretation, agent-role proposal, knowledge synthesis, and conversational responses. Prompts, model identifiers, sampling parameters, and repair attempts are versioned. No language-model output is accepted as structurally valid without schema and invariant checks. The reported benchmark neither invokes a model nor uses one as a semantic judge. Editorial review services are not part of the system or the scientific evaluation.

4 Evaluation Method

4.1 Scope and study design

We conduct a software-system evaluation of the interaction contract, its runtime enforcement, and its authoring controls. The evaluation does not score generated answers or human experience. It has three parts: (1) an exhaustive structural routing benchmark over the authored case corpus, (2) contract tests that execute valid and deliberately broken Unity/backend traces, and (3) transaction tests for the placement-authoring loop.

The structural benchmark compares two executable adapters. The *direct-artifact* adapter reads the scene-semantics, agent-role, and handoff artifacts used by the existing implementation. The *model-grounded* adapter reads a fresh native

FunctionalMLDS instance regenerated in memory from the corresponding version-0.5 case model. Both adapters expose the same normalized operation: resolve the owner of an object and classify a route from a given starting agent as local, directly permitted, transitively reachable, or unreachable. This is an adapter-level structural comparison, not a comparison of conversational models or Unity frame rates. The adapters extract their ownership candidates and handoff graphs separately, but one shared routing function classifies both normalized representations. Agreement therefore measures representation and serialization parity under a fixed policy; it is not an independent validation of the routing algorithm.

4.2 Cases and fixed generation inputs

The corpus comprises all three spatial scene descriptions available in the repository: a career-fair room, a classroom with a dinosaur exhibit, and a food or trade-fair room. They were created as development cases rather than sampled from a room population. They provide 27, 36, and 30 routable scene objects and 5, 4, and 6 agents, respectively. The object universe is the union of the agents' explicitly grounded object identifiers. One zone-only semantic reference in the food/trade-fair case is disclosed and excluded because it is not an executable agent-grounding target.

The frozen corpus contains eight successful structured-output calls to gpt-4.1-mini-2025-04-14: two for scene semantics, three for agent roles and handoffs, and three for knowledge; a ninth semantics artifact was recovered deterministically. Calls used temperature 0.2, stage limits of 1,800, 2,200, or 3,200 output tokens, and one attempt despite configured repair allowances; the provider exposed no seed. The authors inspected and froze the outputs, which are case inputs rather than independent correctness labels. Placement, assembly, comparison, mutation, and metrics are then deterministic and API-free. A coding assistant supplied neither labels nor metric values.

4.3 Responsibility and routing probes

Both adapters apply the same precedence rule: an explicit asset assignment outranks a matching object-group assignment, which outranks a matching zone. More than one owner at the highest active tier is ambiguous and must fail closed. For every routable object we first compare the resolved owner between adapters. We then pair that object with every possible starting agent in its case and compare the route classification. The resulting corpus is exhaustive for these three fixed models; it is not sampled from a larger population.

Natural inputs assign every routable object at the asset tier. We therefore keep fallback tests separate from natural-corpus results. For one copied object per case, we derive three synthetic probes: remove the asset assignment to exercise group fallback, also remove the group assignment to exercise zone fallback, and add a competing owner at the highest active tier to exercise ambiguity rejection. The same normalized candidate sets are supplied to both adapters. Because the direct-artifact source has no native object-group relationship, these probes test the decision rule, not equivalent native authoring expressiveness.

4.4 Fault injection and edit counts

The common mutation denominator contains three semantically equivalent changes per case: remove ownership of an object, assign a second owner, and introduce a dangling handoff. For each adapter and mutation we start from an accepted copy, apply a deterministic patch, and record whether a new validation error is raised and whether it identifies the affected relationship. We also count edited top-level artifacts and changed references. These counts describe the scripted representation patches; they are not elapsed developer effort.

A separate V2-only suite removes a capability provider, duplicates a source agent identifier, or reassigns a domain capability to the runtime orchestrator. We report the canonical model validator and the stricter pipeline provider contract separately, because combining them would obscure which rule detected a fault. Expected-valid copies measure false positives. For reproducibility, each case uses the lexicographically first two routable assets for the missing- and duplicate-owner mutations and the first sorted handoff edge for the dangling-reference mutation. A *reference edit* is one added, removed, or replaced stored identifier; an *artifact edit* means that at least one such change occurred in that top-level representation file. A detection is localized when a new validator issue names a mutation-specific identifier or field token.

4.5 Local structural runtime

For descriptive overhead, each adapter repeatedly constructs its normalized responsibility view and enumerates all object-by-start-agent routes. After four warm-ups, we alternate adapter order for 40 repetitions per case and measure with a monotonic nanosecond clock. JSON input/output, version-0.5-to-V2 generation, Unity, and language-model latency are excluded. We report medians with interquartile and minimum–maximum ranges and the V2/direct median ratio, but do not infer a population-level performance advantage.

4.6 Runtime contract tests

Backend tests pin a V2 model at session setup and exercise deictic and non-deictic chat requests. Negative cases include a missing spatial context, an unknown entity, an obsolete model hash, an ambiguous target, an invalid active agent, and an unmodeled handoff. Each test snapshots session history and checks that a rejected request caused no mutation. Cases decidable from the request and pinned contract additionally require zero response-model calls. The unmodeled handoff is observable only after the stub proposes a destination, so that case instead checks transactional rollback after one stub call. Positive cases check the server-derived entity, group, zone, responsible agent, evidence references, routing reason, and modeled binding returned to Unity. These tests replace the online response model with a deterministic structured stub. The stub lets the test count calls; its sentence is not evaluated as a natural-language answer.

An additional exhaustive integration sweep converts every one of the 459 asset–start-agent routing probes into a backend request. Local and direct routes are expected to be admitted; transitive and unreachable routes are expected to fail because the online contract permits at most one modeled direct handoff. For each admitted request, the test requires the expected target, routed provider, and non-empty capability and runtime identifiers to agree across the checked response fields. For each rejected request, it requires rejection before the stub and no session mutation. The suite blocks network access, counts deterministic-stub calls, and restores the session snapshot after each probe. Expected route class and provider come from the direct-artifact adapter. Binding identifiers are checked for agreement among the runtime action, top-level response, grounding, and routing records produced from the same pinned model. The sweep consequently tests contract enforcement and evidence preservation against the frozen artifacts; it is not an external semantic oracle for the generated binding identifiers.

Unity batch-mode smokes execute the same interaction-evidence component used by the client. A valid target and route must reach completion. A mismatched response entity or improper grounding claim must produce a failed assessment, while a missing deictic context or ambiguous selection must be blocked before serialization. A modeled handoff must pass its permission probe, and mutated trace references must fail closed. Non-deictic interactions are represented explicitly and cannot claim spatial grounding evidence. A separate multi-scenario bridge smoke loads the generated model through the same runtime bridge used by the client, selects an exact target-specific scenario,

checks the server-confirmed binding, and verifies that scene bootstrap uses unique stable-object matches rather than UUID-suffix variants.

4.7 Placement-authoring transaction

The editor’s authoring test begins from a validated FunctionalMLDS draft. It inspects the current revision, previews a structured placement change without writing files, applies the change through the deterministic placement and model stages, and then either accepts or discards it. A subsequent undo restores the last accepted snapshot without restarting the backend. Tests compare before/after values and artifact bytes, reject stale revisions and unknown agents, force a regeneration failure to exercise rollback, and ensure that a runtime project cannot be committed while a change awaits acceptance. Responsibility editing is not evaluated or presented as implemented.

4.8 Measures, analysis, and reproducibility

For RQ1 we report complete asset-specific chains from scenario and capability use through provider, binding, action, assertion, and validation case; unique, ambiguous, and unassigned ownership; route classes; one-hop coverage; graph reachability; adapter parity; and the separate fallback probes. For RQ2 we report mutation detection, false positives, localization, representation edits, and descriptive local runtime. For RQ3 we report pass/fail outcomes for request validation, scenario evidence, rollback, discard, and undo.

A chain is complete only when one target asset resolves through a provider-coherent scenario step and capability use to an executable binding, action, all resulting assertions, and a validation case that covers the same use case, binding, and assertions. One-hop coverage counts local-owner and direct-handoff routes; graph reachability additionally counts transitive paths. The chain denominator counts declared model pairs: it is a structural completeness measure, not a count of conversations or user interactions that were executed. Transitive reachability is an offline structural property, not a claim that the online runtime executes a multi-hop handoff in one turn.

All proportions include their numerator and denominator. We do not attach confidence intervals or significance tests to exhaustive deterministic probes over three purposively authored cases: such intervals would imply a sampling model the design does not support. Repetitions characterize local timing noise only. A single API-free command records software versions, source and input hashes, immutable JSON results, and generated tables. The reported evaluation contains no human participants or personal data and supports no claims about usability, preference, trust, or workload. The anonymous supplement provides the frozen case inputs, both adapters, mutation definitions, raw results, data dictionary, and verifier needed to inspect and rerun these measurements.

5 Results

All structural values below were recomputed by the API-free benchmark from the three frozen cases. They are exact counts for this corpus rather than estimates of a room population. The backend tests used a deterministic response stub, and the Unity values are editor batch-mode results. No result in this section comes from a human-participant study or measures answer quality.

5.1 RQ1: Representation and routing conformance

Table 1 reports the target-specific chains. Every one of the 93 scene assets occurred in at least one complete chain. All 186 ScenarioStep/CapabilityUse pairs that explicitly targeted an asset resolved through a provider, runtime binding

Table 1. Complete model-grounded interaction chains. “Pairs” are asset-targeting ScenarioStep/CapabilityUse pairs.

Case	Assets	Complete	Pairs	Complete
Career fair	27	27	54	54
Classroom	36	36	72	72
Food/trade fair	30	30	60	60
Total	93	93	186	186

Table 2. Offline structural routing from every start agent. “Reach.” includes transitive graph paths; it is not online task completion.

Case	<i>n</i>	Local	Direct	Trans.	Unreach.	Reach.
Career fair	135	27	53	55	0	135
Classroom	144	36	81	9	18	126
Food/trade fair	180	30	71	79	0	180
Total	459	93	205	143	18	441

and action, resulting assertions, and a covering validation case; the checker reported no incomplete chain. The two chains per asset encode the modeled answer and handoff paths, not two observed visitor interactions.

On the natural corpus, responsibility resolution was unique at the asset-assignment tier for 93/93 assets; none required group or zone fallback, and none was ambiguous or unassigned. We therefore do not present fallback as a natural-corpus frequency. In the separate derived priority suite, both adapters produced the expected result for all nine decision cases (9/9 per adapter, 18/18 adapter outcomes): three group fallbacks, three zone fallbacks, and three fail-closed ambiguities.

The routing benchmark paired each asset with every possible starting agent. Table 2 shows all 459 decisions. The owner was already local in 93 cases, a direct handoff was declared in 205, a route existed only transitively in 143, and 18 were unreachable. Thus local or direct one-hop coverage was 298/459 (64.9%), while offline graph reachability was 441/459 (96.1%). The latter number does not mean that the online interface performs a multi-hop handoff in one turn. The direct-artifact and model-grounded adapters agreed on all 93 owner decisions and all 459 route classifications under the shared deterministic resolver.

5.2 RQ2: Representation tradeoffs

The natural, unmodified representations had no owner or route mismatch. Agreement also remained exact after each of the nine semantically matched mutations (three mutation types in each of three cases). Table 3 separates validation from representation edits. Each adapter accepted all three expected-valid case copies, so neither produced a false positive. Each detected and localized 9/9 common mutations: missing ownership, a duplicate owner, and a dangling handoff in every case. This result establishes parity for the injected fault set; it does not show greater fault sensitivity for either representation.

The edit counts expose a mixed representation tradeoff. Across the nine patches, the model-grounded condition changed fewer top-level artifacts (9 versus 15) but more stored references (18 versus 15). These are automatic patch deltas, not evidence that a person would complete a maintenance task faster or with fewer errors. For the additional V2-only mutations, the canonical validator detected 3/9 and the stricter agent-provider contract detected 9/9; each

Table 3. Common-fault validation and deterministic patch counts. Artifact and reference edits describe scripted representation changes, not developer actions or time.

Adapter	FP	Detected	Localized	Artifacts	Refs.
Direct artifact	0/3	9/9	9/9	15	15
Model grounded	0/3	9/9	9/9	9	18

Table 4. Local structural runtime over 40 repetitions per adapter and case, in milliseconds: median [Q1, Q3] (minimum–maximum).

Case	Direct artifact	Fresh V2	Median ratio
Career fair	0.744 [0.693, 0.998] (0.653–1.575)	4.068 [3.976, 4.577] (3.581–7.796)	5.47
Classroom	0.760 [0.724, 0.794] (0.683–1.638)	4.868 [4.791, 5.726] (4.661–8.433)	6.41
Food/trade fair	1.776 [1.661, 1.928] (0.990–2.286)	8.298 [7.895, 8.712] (5.559–10.797)	4.67

localized every fault it detected and yielded 0/3 false positives on expected-valid copies. Because the direct artifacts have no equivalent constructs, these V2-only outcomes indicate added contract expressiveness, not a baseline failure.

Table 4 reports the local structural workload. The fresh V2 adapter took 4.67–6.41 times the direct-adapter median across cases. Absolute medians remained below 8.3 ms, but the measurement excludes parsing, V2 generation, Unity, networking, and language-model latency. It therefore characterizes only the in-memory adapter workload and is not an end-user response-time result.

5.3 RQ3: Contract enforcement and placement transactions

The exhaustive runtime integration sweep passed all 459/459 probes. It admitted all 298/298 local and direct routes, preserved the expected target and provider, returned non-empty capability and runtime identifiers consistently across all checked response fields, and made exactly 298 deterministic-stub calls. It rejected all 161/161 transitive and unreachable routes before the stub with no session mutation. The per-case probe totals were 135, 144, and 180; all session states were restored and no network or external API was used. These probes execute the request contract, but they are not 459 human interactions.

The targeted backend suite accepted the valid deictic request and returned the server-resolved spatial, provider, route, binding, action, and evidence fields. It rejected all 16 invalid variants: five stale, unknown, ambiguous, or contradictory contexts; one unmodeled handoff; one targetless request without an exact action; three invalid interaction-mode/context combinations; one spatial context on a non-deictic request; one invalid active agent; and four oversized fields. Session state remained unchanged. Fifteen failed before the stub; the unmodeled handoff required one call to observe the prohibited destination and then rolled back. These are contract tests, not dialogue-quality trials.

All three Unity Editor batch entry points returned zero with explicit success markers. The native V2 smoke covered scenario progression, dispatch, five assertion types, validation recording, and malformed-model rejection. The bridge wrote 8/8 events and 8/8 validation records; its five assessments yielded the expected inconclusive, pass, and fail outcomes, while three transport events remained inconclusive. The spatial smoke covered ambiguous registries, stable scene binding, serialization, desktop-ray selection, and unsupported-modality rejection. The Steinpilz multi-scenario smoke loaded 31 contexts and 73 actions, selected an exact target-specific chain, accepted the server-confirmed binding, and uniquely bound 30/30 stable objects without UUID-suffix fallback. These are editor batch paths, not human, headset, or frame-rate evidence.

Finally, all four placement-authoring transaction workflows passed. Preview left files and session placement unchanged and blocked project commit while a decision was pending; apply-accept-undo restored the original bytes; a stale revision was rejected and discard restored the original files; and an injected regeneration failure rolled back both files and session state while retaining the preview for correction. Detection and recovery are stated separately: the stale revision and injected regeneration fault were each detected (2/2), and the corresponding non-destructive conflict handling and rollback both completed (2/2). No responsibility, capability, or handoff authoring is implied by these placement-only results.

6 Discussion

6.1 Implications for intelligent-interface authoring

Agent behavior in a spatial interface begins with a user-facing task and environmental referent, not the chat endpoint. The explicit path gives developers a pre-interaction view of responsibility, permitted behavior, expected outcomes, and failures such as unknown targets, ambiguous owners, disallowed handoffs, missing bindings, or contradictory observations. This is a diagnostic and enforcement claim, not an explanation of model reasoning. It is most useful for evolving scenes with divided agent responsibility; a static single-chatbot interface may not justify the added representation. The workbench currently revises placement, not responsibility or capabilities.

6.2 Control and failure recovery

An intelligent interface should communicate when it cannot ground a request or when no agent has the required capability. Silent fallback to a generic agent can produce a fluent but misleading response. The current visitor path exposes no-target and ambiguity status and blocks the answer transition; route and assertion diagnostics are primarily developer-facing. Turning these states into well-designed clarification, transparent-handoff, or abstention dialogues is future interaction work.

6.3 Practical and societal considerations

Spatial agent systems can encode sensitive information about rooms, user locations, speech, and interaction histories. A trace that improves debugging can also increase surveillance risk. Deployments should minimize retained location and conversation data, separate technical identifiers from personal identities, and make retention visible to users.

In the prototype, a request sends the utterance together with entity and source identifiers, ray-hit coordinates, distance, and input modality to the backend. For an online reply, the model provider receives the utterance, bounded history, and trusted entity, group, and zone context, but not the raw ray coordinates. The backend keeps that bounded history in memory. Persistent runtime traces record model, scenario, target, provider, action, status, and utterance length rather than the utterance text. This data minimization is useful but incomplete: the prototype has no visitor-facing consent, retention, export, or deletion controls.

Agent roles and knowledge assignments can reproduce organizational bias. A model makes assignments inspectable but does not make them fair. Authors still need procedures for reviewing who is represented, which agent is considered authoritative, and which requests are refused or redirected. Likewise, language-model safeguards and knowledge provenance remain necessary even when the surrounding configuration is valid.

6.4 When a model helps—and when it does not

The approach trades authoring structure for validation and runtime evidence and helps only while model and runtime remain synchronized. Both adapters detected the common fault set; higher detection was therefore not the observed benefit. Model-grounded patches touched fewer top-level artifacts and expressed added provider constraints, but changed more references and had higher local structural runtime. Whether this trade helps authors is an empirical interaction question.

7 Limitations and Threats to Validity

The three environments are a purposive convenience corpus, two are exhibition-like, and every natural-corpus object has an asset-level owner. Group and zone fallback therefore use synthetic copied candidates. Results establish behavior for these models, not population frequencies.

The comparison holds meaning constant but covers neither complete applications nor human authoring. Its three common mutations and scripted edit counts do not measure developer effort or unknown faults. Both representations derive from the same semantic sources, and one resolver classifies both normalized graphs; parity is a controlled representation check, not independent routing evidence or proof of architectural superiority. Likewise, the runtime sweep takes route and provider expectations from the direct adapter, while binding checks compare fields derived from one pinned model. These are consistency checks, not independent semantic oracles.

We evaluate neither answer quality nor usability, trust, workload, or preference. The server checks entity, route, and evidence but cannot prove that client-reported ray geometry is physical; deployment needs a trusted client and transport boundary. Unity evidence is limited to desktop selection and Editor batch paths, with no headset or WebXR result. Authoring covers placement, not responsibility, capability, or handoff editing.

The V2 assembler and validator share a codebase, so their agreement is not independent standards conformance; full EAST-ADL conformance is not claimed. Timing covers in-memory work on one Windows machine and excludes parsing, generation, Unity, network, and model latency. Finally, frozen semantic inputs came from model calls without a provider seed: stored responses are replayable, but service generation is not bit-for-bit reproducible. Structural measurements are deterministic and API-free; editorial feedback supplies no evaluation evidence.

8 Conclusion

We presented an interaction-model-grounded assurance layer for embodied multi-agent interfaces in Unity. Rather than replacing interaction design with a metamodel, it requires each admitted request to follow one pinned chain from spatial target through agent capability and runtime action, while a rejection identifies the violated relation.

All 93 assets had complete provider-coherent chains, and all 186 declared target-specific candidates were structurally complete. Under a shared resolver, the adapters agreed on 93 owner and 459 route decisions and detected all nine common injected faults. The runtime sweep admitted all 298 local or direct routes with consistent binding evidence and rejected all 161 transitive or unreachable routes before the response stub without session mutation. Unity batch tests exercised 31 scenario contexts and 73 actions; four placement workflows preserved preview and recovery guarantees.

These corpus-bounded results support FunctionalMLDS as an executable assurance boundary, not claims of better answers, usability, trust, or authoring productivity. Whether the added structure helps people design or understand spatial-agent interactions remains a human-participant question.

GenAI Usage Disclosure

The case-study pipeline made eight successful structured-output calls to OpenAI gpt-4.1-mini-2025-04-14 at temperature 0.2 to propose two scene-semantics artifacts, three agent-role artifacts with handoff declarations, and three sets of local knowledge entries. The remaining scene-semantics artifact was recovered deterministically without a model call. The prompts, returned JSON, model identifiers, token metadata, and validation records are stored with the case artifacts. No repair call was needed for the fixed outputs used in the evaluation; the API did not expose a sampling seed. The authors reviewed these artifacts and then froze them as inputs. Evaluation utterances, model assembly, fault injection, measurements, and tables are deterministic, and the reported structural benchmark made no model or network calls. No generative model was used as a semantic answer-quality judge.

OpenAI Codex, for which the available interface identified the GPT-5 model family rather than a more specific deployment identifier, assisted with prototype hardening, test and analysis code, literature-search support, and drafting and editing this manuscript. The authors inspected and revised the resulting code, verified cited sources and generated evidence, and remain accountable for all claims and prose. The paperreview.ai reviewer service was used only for editorial feedback on submission drafts and did not supply scientific evidence or reported measurements.

A Detailed Positioning and Enforcement Matrices

Table 5 spells out the model objects used by the running example in Section 3.

Table 5. Running example: representing “What is this?” while a visitor selects the classroom dinosaur skeleton.

Element	Example instance	Meaning
Target	Dinosaur skeleton, its group, and exhibit zone	Stable referent selected in Unity
Scenario step	Describe selected dinosaur skeleton	User-visible interaction to execute
Capability use	Describe room object; provider: Exhibit Interpreter	One occurrence with an explicit provider and targets
Capability	Describe room object	Declared operation that the provider may perform
Agent	Exhibit Interpreter	Domain agent responsible for the asset
Runtime action	POST /chat	Backend operation permitted by the binding
Assertion	Target, provider, response entity, and route agree	Observable pass/fail evidence
Validation case	Grounded response contract probe	Links the use case, binding, assertions, and verdict

Table 6 expands the closest-work positioning in Section 2. It compares each contribution’s stated primary unit and emphasis; an empty cell is not evidence that a cited system lacks an unlisted mechanism.

Table 7 expands the staged enforcement account in Section 3. The backend remains authoritative even when the client performs an earlier equivalent check.

B Supplementary Evaluation and Artifact Checklist

The anonymous supplement contains:

- immutable scene and agent inputs;
- the fixed authored probe corpus and expected structural outcomes;
- direct-artifact and model-grounded condition definitions;
- change-task patches and fault-mutation catalog;

Table 6. Closest-work comparison by primary represented unit and assurance locus.

Work	Represented unit	Interface/runtime emphasis	Distinction of this work
LLMR [12]	Plans and edits to a mixed-reality world	Prompting, scene understanding, execution, and self-debugging	We constrain a declared path through target, provider, capability, and action instead of generating worlds.
Tell-XR [9]	Event-condition-action automation	Conversational end-user development of XR behavior	We study responsibility and evidence across a multi-agent interaction, not automation-language expressiveness.
Campos-López et al. [8]	Domain concepts, AR representations, and external interactions	Model-to-application generation and user evaluation	Our model remains active as an admission and diagnostic boundary after scene materialization.
Karre and Reddy [17]	VR requirements and role-based scene/action/state templates	Requirements and mappings to VR application elements	We specialize the runtime link to assets, agent authority, actions, and observed assertions.
FunctionalMLDS	Scenario, target, provider, capability, action, and assertion contract	Unity selection, fail-closed routing, and relation-level evidence	The contribution is a non-bypassable composed interaction guarantee and failure localization, not a general XR authoring language.

Table 7. Invariant-enforcement layers in the implemented interaction path.

Layer	Representative invariant	Failure boundary
JSON schema	Required fields, value types, enumerations, and reference shapes	Before constructing a model instance.
Canonical V2 model	Referential closure, typed cardinalities, assertion/VV ownership, and no direct step-to-action shortcut	The instance is rejected before materialization.
Executable profile	One main scenario, one provider, provider/performer/capability agreement, and target coherence	Only chains surviving every stage validator are published.
Backend runtime	Pinned hash, trusted entity, derived group and zone, exact target/provider/ action mapping, permitted handoff, and request/response schemas	Before session mutation and, when request-decidable, before a model call.
Unity client	Stable collider binding, explicit unresolved states, and returned model/target/provider/route/action agreement	Before serialization or answer presentation; backend checks remain authoritative.

- raw machine-readable outcomes;
- scripts that regenerate tables and figures; and
- environment and content-hash manifests.

Repository URLs, file metadata, screenshots, and videos supplied for review are checked for identifying information.

References

- [1] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, 1–13. doi:10.1145/3290605.3300233
- [2] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. 2021. Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, 1–16. doi:10.1145/3411764.3445717
- [3] Timothy Bickmore, Laura Pfeifer, and Daniel Schulman. 2011. Relational Agents Improve Engagement and Learning in Science Museum Visitors. In *Intelligent Virtual Agents*. Springer, 55–67. doi:10.1007/978-3-642-23974-8_7
- [4] Kadir Burak Buldu, Süleyman Özdel, Ka Hei Carrie Lau, Mengdi Wang, Daniel Saad, Sofie Schönborn, Auxane Boch, Enkelejda Kasneci, and Efe Bozkir. 2025. CUIfy the XR: An Open-Source Package to Embed LLM-Powered Conversational Agents in XR. In *2025 IEEE International Conference*

- on Artificial Intelligence and eXtended and Virtual Reality. IEEE, 192–197. doi:10.1109/AIXVR63409.2025.00037
- [5] Louis Burk, Alexander Fischer, Uwe Wienkop, Ramin Tavakoli Kolagari, Christoph Scharnagl, and Alexandra Arzberger. 2026. Handling Toolchain Evolution with Modular Meta-Languages: An Approach for Structured LLM-Based Artifact Generation. In *Proceedings of the 21st International Conference on Evaluation of Novel Approaches to Software Engineering*, Vol. 1. SciTePress, 807–814. doi:10.5220/0014715200004015
- [6] Louis Burk and Uwe Wienkop. 2025. Dynamische Optimierung von VR-Lernumgebungen durch generative KI. In *Zukunft MINT Lehre: Was bleibt? Was kommt? Was wirkt? Tagungsband zum 6. Symposium zur Hochschullehre in den MINT-Fächern (Didaktiknachrichten)*, Hanna Dölling and Claudia Schäfle (Eds.). BayZiel – Bayerisches Zentrum für Innovative Lehre, München, 300–308. DiNa-Sonderausgabe, ISSN 1612-4537. https://bayziel.de/wp-content/uploads/Tagungsband_MINTSymposium_2025.pdf
- [7] Gaëlle Calvary, Joëlle Coutaz, David Thevenin, Quentin Limbourg, Laurent Bouillon, and Jean Vanderdonckt. 2003. A Unifying Reference Framework for Multi-target User Interfaces. *Interacting with Computers* 15, 3 (2003), 289–308. doi:10.1016/S0953-5438(03)00010-9
- [8] Rubén Campos-López, Esther Guerra, Juan de Lara, Alessandro Colantoni, and Antonio Garmendia. 2023. Model-Driven Engineering for Augmented Reality. *Journal of Object Technology* 22, 2 (2023), 2:1–2:15. doi:10.5381/jot.2023.22.2.a7
- [9] Alessandro Carcangiu, Marco Manca, Jacopo Mereu, Carmen Santoro, Ludovica Simeoli, and Lucio Davide Spano. 2025. Tell-XR: Conversational End-User Development of XR Automations. In *Human-Computer Interaction – INTERACT 2025*. Springer, 617–641. doi:10.1007/978-3-032-04999-5_35
- [10] Justine Cassell, Timothy W. Bickmore, Mark Billinghurst, Lee Campbell, Kenny Chang, Hannes Högni Vilhjálmsson, and Hao Yan. 1999. Embodiment in Conversational Interfaces: Rea. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 520–527. doi:10.1145/302979.303150
- [11] Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. 2026. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 19888–19905. doi:10.18653/v1/2026.acl-long.912
- [12] Fernanda De La Torre, Cathy Mengying Fang, Han Huang, Andrzej Banburski-Fahey, Judith Amores Fernandez, and Jaron Lanier. 2024. LLMR: Real-time Prompting of Interactive Worlds using Large Language Models. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, 1–22. doi:10.1145/3613904.3642579
- [13] EAST-ADL Association. 2013. *EAST-ADL Domain Model Specification*. Language Specification Version 2.1.12. EAST-ADL Association. https://east-adl.info/Specification/V2.1.12/EAST-ADL-Specification_V2.1.12.pdf
- [14] Alexander Fischer, Louis Burk, Ramin Tavakoli Kolagari, and Uwe Wienkop. 2025. Machine-Readable by Design: Language Specifications as the Key to Integrating LLMs into Industrial Tools. In *Proceedings of the 20th Conference on Computer Science and Intelligence Systems*, Vol. 43. Polish Information Processing Society, 531–542. doi:10.15439/2025F5613
- [15] Peter Gorniak and Deb Roy. 2005. Probabilistic Grounding of Situated Speech Using Plan Recognition and Reference Resolution. In *Proceedings of the 7th International Conference on Multimodal Interfaces*. ACM, 138–143. doi:10.1145/1088463.1088489
- [16] Orlena C. Z. Gotel and Anthony C. W. Finkelstein. 1994. An Analysis of the Requirements Traceability Problem. In *Proceedings of the First IEEE International Conference on Requirements Engineering*. IEEE, 94–101. doi:10.1109/ICRE.1994.292398
- [17] Sai Anirudh Karre and Y. Raghu Reddy. 2024. Model-based Approach for Specifying Requirements of Virtual Reality Software Products. *Frontiers in Virtual Reality* 5 (2024), 1471579. doi:10.3389/frvir.2024.1471579
- [18] Casey Kennington and David Schlangen. 2015. Simple Learning and Compositional Application of Perceptually Grounded Word Meanings for Incremental Reference Resolution. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 292–301. doi:10.3115/v1/P15-1029
- [19] Thomas Kollar, Stefanie Tellex, Deb Roy, and Nicholas Roy. 2010. Toward Understanding Natural Language Directions. In *2010 5th ACM/IEEE International Conference on Human-Robot Interaction*. IEEE, 259–266. doi:10.1109/HRI.2010.5453186
- [20] Stefan Kopp, Lars Gesellensetter, Nicole C. Krämer, and Ipke Wachsmuth. 2005. A Conversational Agent as Museum Guide: Design and Evaluation of a Real-World Application. In *Intelligent Virtual Agents*. Springer, 329–343. doi:10.1007/11550617_28
- [21] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. 2013. *PROV-O: The PROV Ontology*. W3C Recommendation. World Wide Web Consortium. <https://www.w3.org/TR/2013/REC-prov-o-20130430/>
- [22] Séverin Lemaignan, Raquel Ros, E. Akin Sisbot, Rachid Alami, and Michael Beetz. 2012. Grounding the Interaction: Anchoring Situated Discourse in Everyday Human-Robot Interaction. *International Journal of Social Robotics* 4, 2 (2012), 181–199. doi:10.1007/s12369-011-0123-x
- [23] Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *The Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303. doi:10.1016/j.jlap.2008.08.004
- [24] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, Vol. 36. 51991–52008. doi:10.52202/075280-2264
- [25] Q. Vera Liao, Daniel Gruen, and Sarah Miller. 2020. Questioning the AI: Informing Design Practices for Explainable AI User Experiences. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, 1–15. doi:10.1145/3313831.3376590
- [26] Quentin Limbourg, Jean Vanderdonckt, Benjamin Michotte, Laurent Bouillon, and Victor López-Jaquero. 2005. USIXML: A Language Supporting Multi-path Development of User Interfaces. In *Engineering Human Computer Interaction and Interactive Systems*. Springer, 200–220. doi:10.1007/11431879_12
- [27] Ziyi Liu, David Li, Zhongyi Zhou, David Kim, Ruofei Du, and Xun Qian. 2026. AgentHands: Generating Interactive Hand Gestures for Spatially Grounded Agent Conversations in XR. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. ACM, 1–24. doi:10.1145/3772318.3790938

- [28] Patrick Mäder and Alexander Egyed. 2015. Do Developers Benefit from Requirements Traceability When Evolving and Maintaining a Software System? *Empirical Software Engineering* 20, 2 (2015), 413–441. doi:10.1007/s10664-014-9314-z
- [29] Bertrand Meyer. 1992. Applying Design by Contract. *Computer* 25, 10 (1992), 40–51. doi:10.1109/2.161279
- [30] Alex Moldovan, Vlad Nicula, Ionut Pasca, Mihai Popa, Jaya Krishna Namburu, Anamaria Oros, and Paul Brie. 2020. OpenUIDL, A User Interface Description Language for Runtime Omni-Channel User Interfaces. *Proceedings of the ACM on Human-Computer Interaction* 4, EICS (2020), 1–52. doi:10.1145/3397874
- [31] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, 1–22. doi:10.1145/3586183.3606763
- [32] Fabio Paternò, Carmen Santoro, and Lucio Davide Spano. 2009. MARIA: A Universal, Declarative, Multiple Abstraction-Level Language for Service-Oriented Applications in Ubiquitous Environments. *ACM Transactions on Computer-Human Interaction* 16, 4 (2009), 1–30. doi:10.1145/1614390.1614394
- [33] Angel R. Puerta and Jacob Eisenstein. 1999. Towards a General Computational Framework for Model-Based Interface Development Systems. In *Proceedings of the 4th International Conference on Intelligent User Interfaces*. ACM, 171–178. doi:10.1145/291080.291108
- [34] Ran Wei, Le Zhu, Haochi Wang, Ruizhe Yang, Jiapeng Guan, Siyuan Ji, Yuchen Hu, Zhe Jiang, and Xiangyang Ji. 2026. Model-Driven Discipline for Multi-Agent LLMs: Requirement-to-Verification Generation of Traceable System Models. arXiv:2607.16708 [cs.SE] doi:10.48550/arXiv.2607.16708
- [35] Justin D. Weisz, Jessica He, Michael Muller, Gabriela Hofer, Rachel Miles, and Werner Geyer. 2024. Design Principles for Generative AI Applications. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, 1–22. doi:10.1145/3613904.3642466
- [36] Qian Yang, Aaron Steinfeld, Carolyn Rosé, and John Zimmerman. 2020. Re-examining Whether, Why, and How Human-AI Interaction Is Uniquely Difficult to Design. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, 1–13. doi:10.1145/3313831.3376301